

Assignment 01

Analysis of Algorithms

Dario Pomasqui

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urcuqui, Imbabura, Ecuador

August 24, 2026

GitHub Repository:

<https://github.com/darioXxx32/Algorithm-analysis/tree/main/assignment01>

The complete source files are available in the repository. This report focuses on the core idea of each activity, so only the most relevant code fragments are shown together with the execution evidence.

1 Exercise 1: Euclid's Algorithm

Euclid's algorithm computes $\gcd(m, n)$ by repeatedly replacing the pair (m, n) with $(n, m \bmod n)$. When the second value becomes zero, the first value is the greatest common divisor.

Analysis and Key Points

The important idea in this exercise is that the GCD does not change when the pair (m, n) is replaced by $(n, m \bmod n)$. Because of that, the code only needs a small loop that keeps moving the remainder forward. The key moment is when the remainder becomes zero; at that point, the last nonzero value is the answer.

Textbook Pseudocode

```
ALGORITHM Euclid(m, n)
while n != 0 do
    r <- m mod n
    m <- n
    n <- r
return m
```

Key Code Fragment

```
int euclid(int m, int n) {  
    while (n != 0) {  
        int r = m % n;  
        m = n;  
        n = r;  
    }  
    return m;  
}
```

The fragment keeps only the essential loop: compute the remainder, shift the two values, and stop when no remainder remains.

Handwritten Work

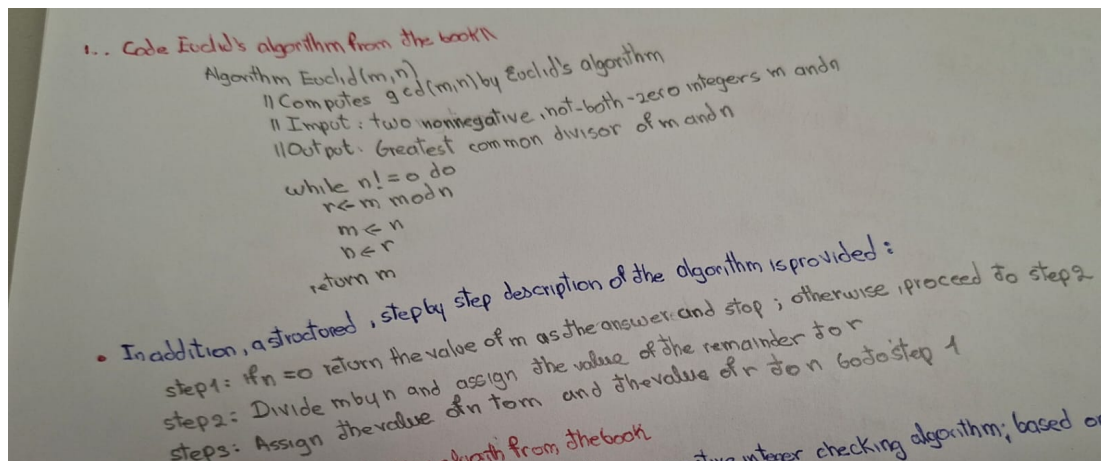


Figure 1: Handwritten work for Euclid's Algorithm.

Execution Output

```
PS C:\Users\User\Desktop\important\reportes> ^C
PS C:\Users\User\Desktop\important\reportes> ./ejer1
Enter m: 60
Enter n: 24
gcd(60, 24) = 12
PS C:\Users\User\Desktop\important\reportes> ./ejer1
Enter m: 31415
Enter n: 14142
gcd(31415, 14142) = 1
PS C:\Users\User\Desktop\important\reportes> ./ejer1
Enter m: 17
Enter n: 31
gcd(17, 31) = 1
PS C:\Users\User\Desktop\important\reportes> 
```

Figure 2: Execution output of Euclid's Algorithm.

2 Exercise 2: Consecutive Integer Checking Algorithm

This algorithm starts from $t = \min\{m, n\}$ and decreases t until it finds a number that divides both inputs. Because it checks candidates from largest to smallest, the first valid divisor is the GCD.

Analysis and Key Points

The key point is that the GCD cannot be larger than the smaller of the two inputs. This method uses that idea directly by testing possible divisors from largest to smallest. It is not as efficient as Euclid's algorithm, but it is very clear: the first value that divides both numbers must be the greatest common divisor.

Textbook-Based Pseudocode

```
ALGORITHM ConsecutiveIntegerChecking(m, n)
t ← min{m, n}
while t > 0 do
    if m mod t = 0 and n mod t = 0 then
        return t
    t ← t - 1
```

Key Code Fragment

```

int consecutiveIntegerChecking(int m, int n) {
    int t = min(m, n);

    while (t > 0) {
        if (m % t == 0 && n % t == 0) {
            return t;
        }
        t--;
    }
    return 1;
}

```

The important part is the descending search. The test `m % t == 0 && n % t == 0` confirms that the current candidate divides both numbers.

Handwritten Work

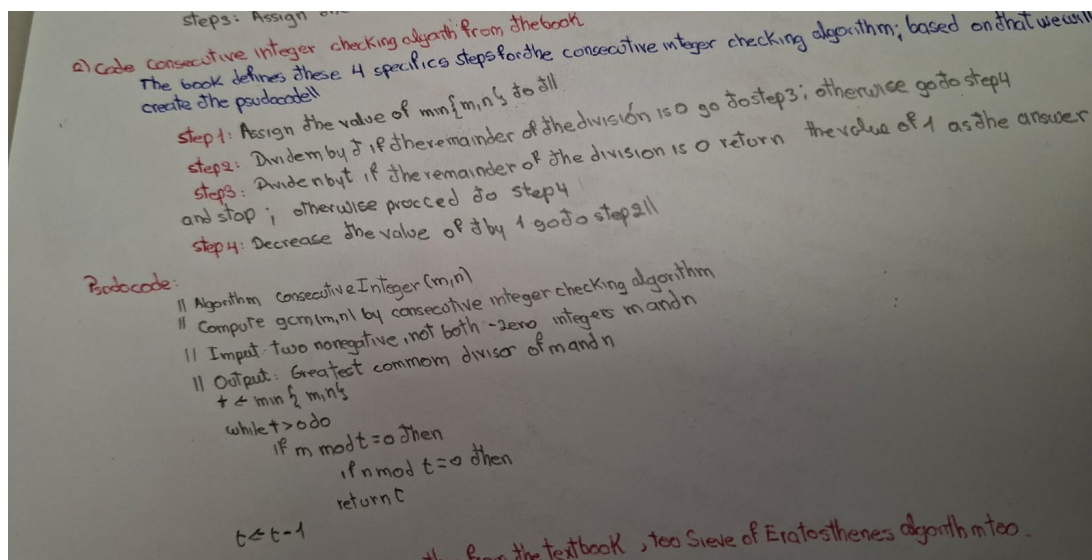
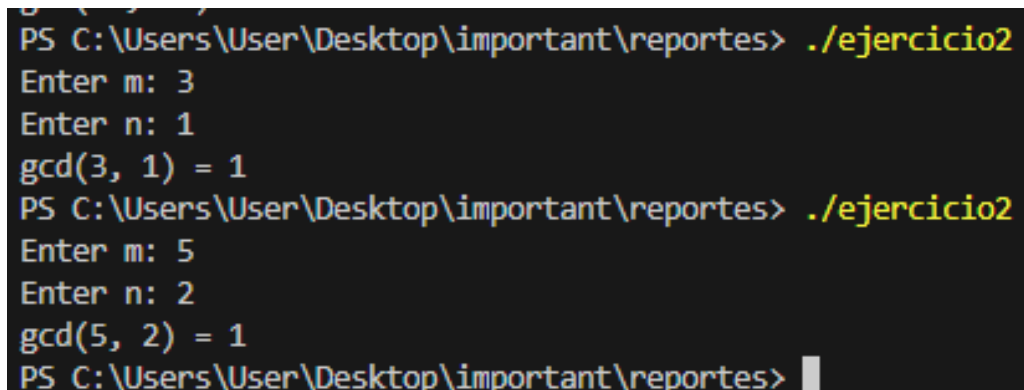


Figure 3: Handwritten work for the Consecutive Integer Checking Algorithm.

Execution Output



```
PS C:\Users\User\Desktop\important\reportes> ./ejercicio2
Enter m: 3
Enter n: 1
gcd(3, 1) = 1
PS C:\Users\User\Desktop\important\reportes> ./ejercicio2
Enter m: 5
Enter n: 2
gcd(5, 2) = 1
PS C:\Users\User\Desktop\important\reportes>
```

Figure 4: Execution output of the Consecutive Integer Checking Algorithm.

3 Exercise 3: Middle-School Procedure and Sieve of Eratosthenes

The middle-school procedure computes the GCD by factoring both numbers into primes, selecting the common prime factors, and multiplying them. The Sieve of Eratosthenes is used first to generate the prime numbers needed for factorization.

Analysis and Key Points

This exercise has more parts than the first two because it combines prime generation, factorization, and comparison. The sieve is useful because it gives the prime numbers needed to factor both inputs. Once both factor lists are ready, the common prime factors can be selected and multiplied, which follows the same logic usually done by hand in the middle-school method.

Core Sieve Fragment

```
for (int p = 2; p * p <= n; p++) {
    if (is_prime[p]) {
        for (int j = p * p; j <= n; j += p) {
            is_prime[j] = 0;
        }
    }
}
```

This loop marks composite numbers as non-prime. Starting at $p * p$ avoids repeating eliminations that were already handled by smaller primes.

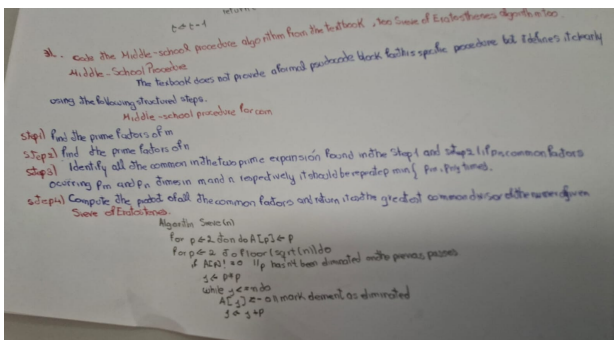
Core GCD Fragment

```
while (i < countM && j < countN) {
    if (factorsM[i] == factorsN[j]) {
        common[countC++] = factorsM[i];
        i++;
        j++;
    } else if (factorsM[i] < factorsN[j]) {
        i++;
    } else {
        j++;
    }
}

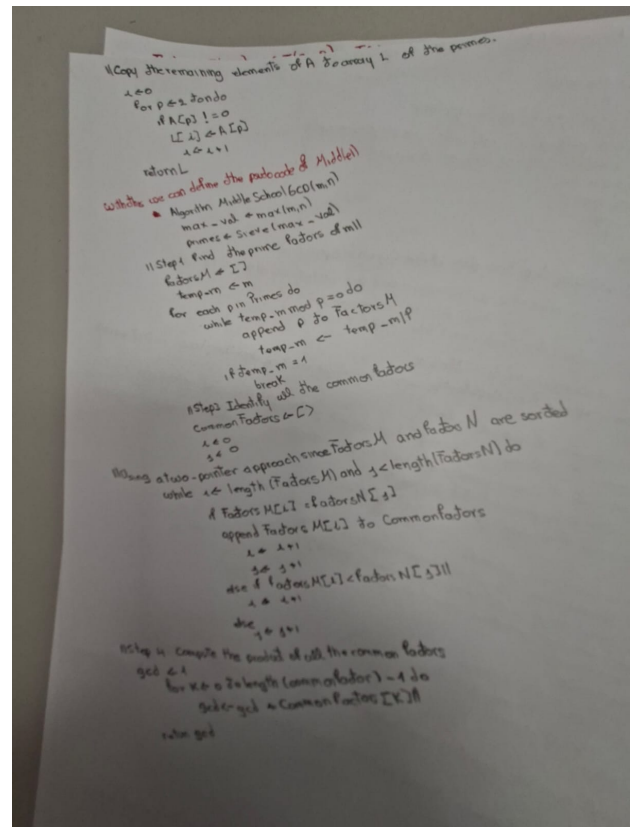
int gcd = 1;
for (int k = 0; k < countC; k++) {
    gcd *= common[k];
}
```

After both numbers are factorized, the two sorted factor lists are compared with two indices. Equal values are common prime factors, and their product gives the final GCD.

Handwritten Work



(a) Part 1



(b) Part 2

Figure 5: Handwritten work for the middle-school procedure and sieve.

Execution Output

```
PS C:\Users\User\Desktop\important\reportes> ./ejercicio3
Enter m: 31415
Enter n: 14142
gcd(31415, 14142) = 1
PS C:\Users\User\Desktop\important\reportes>
```

Figure 6: Execution output of the middle-school GCD implementation.

4 Exercise 4: Stack Behavior Using C++ Lists

A stack follows Last-In, First-Out (LIFO) behavior. With `std::list`, this can be simulated by inserting, reading, and removing elements from the same end of the list.

Analysis and Key Points

The important detail is that a normal list becomes a stack only if its operations are restricted. In this case, everything happens at the back of the list: new elements are inserted there, read there, and removed from there. The output makes the LIFO idea visible because the last value inserted is the first one printed and removed.

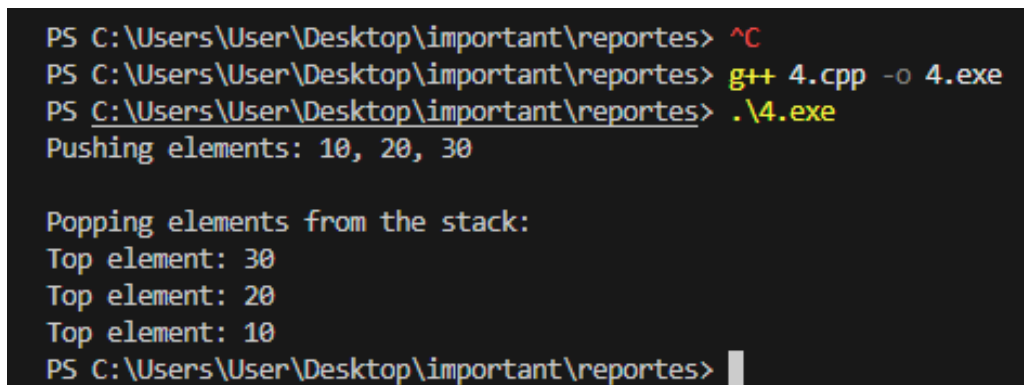
Key Code Fragment

```
stack_list.push_back(10);
stack_list.push_back(20);
stack_list.push_back(30);

while (!stack_list.empty()) {
    cout << "Top element: " << stack_list.back() << endl;
    stack_list.pop_back();
}
```

`push_back()` adds new elements to the top of the simulated stack, `back()` reads the newest element, and `pop_back()` removes it.

Execution Output



```
PS C:\Users\User\Desktop\important\reportes> ^C
PS C:\Users\User\Desktop\important\reportes> g++ 4.cpp -o 4.exe
PS C:\Users\User\Desktop\important\reportes> .\4.exe
Pushing elements: 10, 20, 30

Popping elements from the stack:
Top element: 30
Top element: 20
Top element: 10
PS C:\Users\User\Desktop\important\reportes> █
```

Figure 7: Execution output demonstrating stack behavior with a C++ list.

5 Exercise 5: Custom MyStack Class

The `MyStack` class encapsulates a `std::list` so users can only interact with it through stack operations. This protects the LIFO behavior from arbitrary list modifications.

Analysis and Key Points

This class takes the idea from the previous exercise and makes it safer. The list is private, so the user cannot insert or delete elements in the middle and accidentally break the stack behavior.

The public methods become the controlled interface of the stack, and the empty checks are important because they avoid invalid operations such as reading the top of an empty structure.

Key Code Fragment

```
template <typename T>
class MyStack {
private:
    list<T> elements;

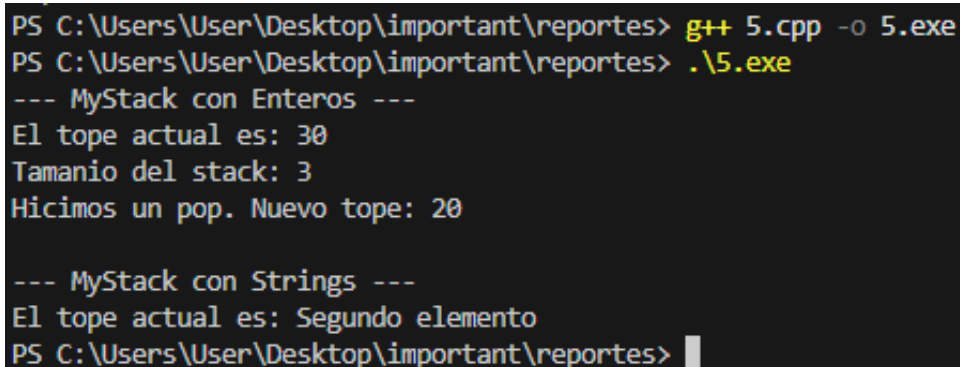
public:
    void push(const T& value) { elements.push_back(value); }

    void pop() {
        if (isEmpty()) throw out_of_range("Stack is empty");
        elements.pop_back();
    }

    T top() const {
        if (isEmpty()) throw out_of_range("Stack is empty");
        return elements.back();
    }
};
```

The private list stores the data, while `push()`, `pop()`, and `top()` expose only the operations that a stack should allow.

Execution Output



```
PS C:\Users\User\Desktop\important\reportes> g++ 5.cpp -o 5.exe
PS C:\Users\User\Desktop\important\reportes> .\5.exe
--- MyStack con Enteros ---
El tope actual es: 30
Tamano del stack: 3
Hicimos un pop. Nuevo tope: 20

--- MyStack con Strings ---
El tope actual es: Segundo elemento
PS C:\Users\User\Desktop\important\reportes> █
```

Figure 8: Execution output of the custom `MyStack` class.

6 Exercise 6: Queue Behavior Using C++ Lists

A queue follows First-In, First-Out (FIFO) behavior. With `std::list`, new elements are inserted at the back and removed from the front.

Analysis and Key Points

For a queue, the essential difference is that insertion and removal happen at opposite ends. New elements enter at the back, while the oldest element leaves from the front. This matches the FIFO idea, and the output confirms it because the first inserted value is also the first one removed.

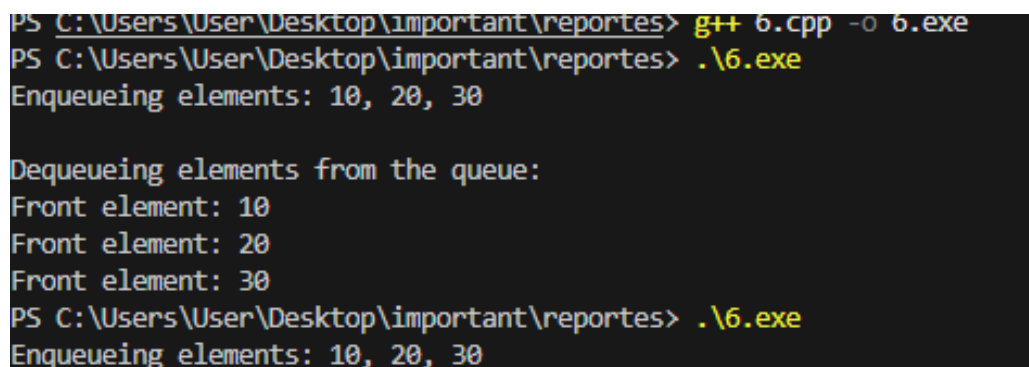
Key Code Fragment

```
queue_list.push_back(10);
queue_list.push_back(20);
queue_list.push_back(30);

while (!queue_list.empty()) {
    cout << "Front element: " << queue_list.front() << endl;
    queue_list.pop_front();
}
```

`push_back()` performs the enqueue operation, while `front()` and `pop_front()` read and remove the oldest inserted element.

Execution Output



```
PS C:\Users\User\Desktop\important\reportes> g++ 6.cpp -o 6.exe
PS C:\Users\User\Desktop\important\reportes> .\6.exe
Enqueueing elements: 10, 20, 30

Dequeuing elements from the queue:
Front element: 10
Front element: 20
Front element: 30
PS C:\Users\User\Desktop\important\reportes> .\6.exe
Enqueueing elements: 10, 20, 30
```

Figure 9: Execution output demonstrating queue behavior with a C++ list.

7 Exercise 7: Custom MyQueue Class

The `MyQueue` class encapsulates queue behavior with a private `std::list`. The public methods enforce FIFO order and include basic exception handling for invalid operations on an empty

queue.

Analysis and Key Points

The main idea is similar to the custom stack, but the direction of removal changes. The private list protects the internal structure, while the public methods force the queue to behave correctly. `push()` adds at the back, and `front()` plus `pop()` work at the front, which keeps the FIFO order consistent.

Key Code Fragment

```
template <typename T>
class MyQueue {
private:
    list<T> elements;

public:
    void push(const T& value) { elements.push_back(value); }

    void pop() {
        if (isEmpty()) throw out_of_range("Queue is empty");
        elements.pop_front();
    }

    T front() const {
        if (isEmpty()) throw out_of_range("Queue is empty");
        return elements.front();
    }
};
```

The queue receives new elements at the back, but returns and removes elements from the front. This is the essential difference from the stack implementation in Exercise 5.

Execution Output

```
Front element: 30
PS C:\Users\User\Desktop\important\reportes> g++ 7.cpp -o 7.exe
PS C:\Users\User\Desktop\important\reportes> .\7.exe
--- MyQueue con Enteros ---
El frente actual es: 10
Tamano de la cola: 3
Hicimos un pop. Nuevo frente: 20

--- MyQueue con Strings ---
El frente actual es: Primer cliente
PS C:\Users\User\Desktop\important\reportes>
```

Figure 10: Execution output of the custom MyQueue class.

8 Exercise 8: C++ Standard Library Stacks and Queues

The C++ Standard Library already provides container adapters for stacks and queues. These adapters restrict the operations of an underlying container, usually `std::deque`, to match a specific data-structure behavior.

Analysis and Key Points

This activity is useful because it shows that C++ already includes ready-made structures for these cases. `std::stack` and `std::queue` are container adapters, so they hide operations that would break LIFO or FIFO behavior. `std::priority_queue` is different because it removes elements by priority, while `std::deque` is more flexible because it works efficiently at both ends.

Key Code Fragments

```
stack<string> myStack;
myStack.push("Primer plato");
myStack.push("Segundo plato");
cout << myStack.top() << endl;
myStack.pop();
```

```
queue<string> myQueue;
myQueue.push("Cliente 1");
myQueue.push("Cliente 2");
cout << myQueue.front() << endl;
myQueue.pop();
```

```
priority_queue<int> pQueue;
pQueue.push(10);
```

```
pQueue.push(100);
pQueue.push(50);
cout << pQueue.top() << endl; // highest priority
```

`std::stack` enforces LIFO order, `std::queue` enforces FIFO order, and `std::priority_queue` removes elements by priority rather than by insertion order.

Execution Output

```
PS C:\Users\User\Desktop\important\reportes> g++ 8.cpp -o 8.exe
PS C:\Users\User\Desktop\important\reportes> .\8.exe
--- 1. std::stack (Pila LIFO) ---
El tope de la pila es: Tercer plato
Tamano de la pila: 3
Sacando: Tercer plato
Sacando: Segundo plato
Sacando: Primer plato

--- 2. std::queue (Cola FIFO) ---
Atendiendo primero a: Cliente 1
El ultimo en la fila es: Cliente 3
Atendiendo a Cliente 1 y retirandolo.
Atendiendo a Cliente 2 y retirandolo.
Atendiendo a Cliente 3 y retirandolo.

--- 3. std::priority_queue (Cola de Prioridad) ---
Elementos en orden de prioridad:
100 50 10

--- 4. std::deque (Double-Ended Queue) ---
Frente: 10, Atras: 30
Queda: 20

PS C:\Users\User\Desktop\important\reportes> 
```

Figure 11: Execution output of the standard library stack, queue, priority queue, and deque examples.

9 Exercise 9: Custom Binary Tree Data Structure

The custom tree was implemented as a Binary Search Tree (BST). In a BST, values smaller than the current node go to the left subtree, while greater values go to the right subtree.

Analysis and Key Points

The key decision was to make the structure a Binary Search Tree, since that gives a clear rule for every insertion. Smaller values move to the left and greater values move to the right.

Recursion fits well here because each child node can be treated like another smaller tree, and the inorder traversal is a good check because a correct BST should print the values in sorted order.

Key Code Fragment

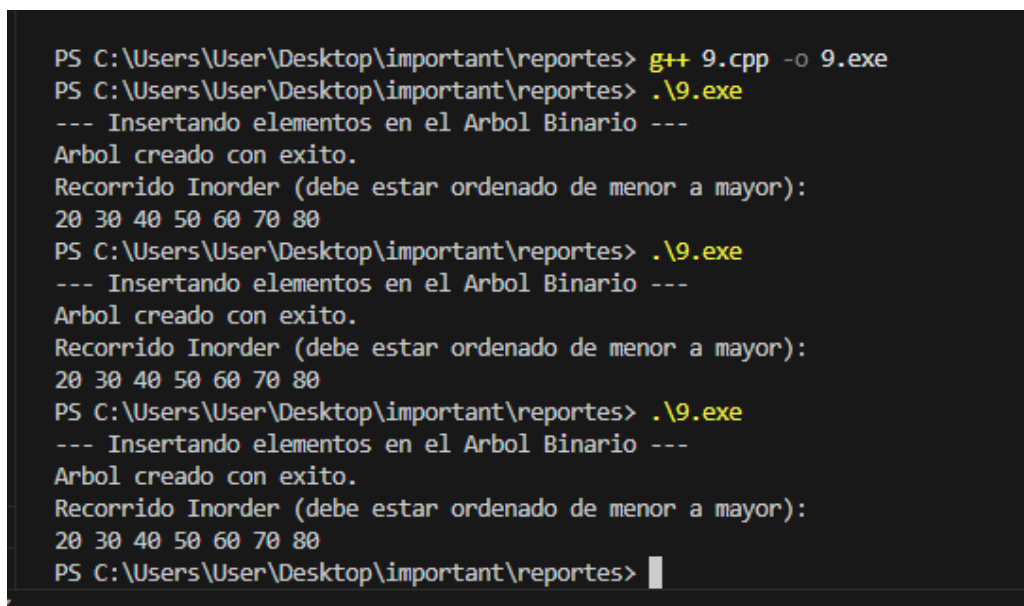
```
Node* insertRecursive(Node* current, int value) {
    if (current == nullptr) {
        return new Node(value);
    }

    if (value < current->data) {
        current->left = insertRecursive(current->left, value);
    } else if (value > current->data) {
        current->right = insertRecursive(current->right, value);
    }

    return current;
}
```

The base case creates a new node when an empty position is found. Otherwise, the value comparison chooses the left or right branch recursively. The inorder traversal was used as validation because a correct BST prints its values in ascending order.

Execution Output



```
PS C:\Users\User\Desktop\important\reportes> g++ 9.cpp -o 9.exe
PS C:\Users\User\Desktop\important\reportes> .\9.exe
--- Insertando elementos en el Arbol Binario ---
Arbol creado con exito.
Recorrido Inorder (debe estar ordenado de menor a mayor):
20 30 40 50 60 70 80
PS C:\Users\User\Desktop\important\reportes> .\9.exe
--- Insertando elementos en el Arbol Binario ---
Arbol creado con exito.
Recorrido Inorder (debe estar ordenado de menor a mayor):
20 30 40 50 60 70 80
PS C:\Users\User\Desktop\important\reportes> .\9.exe
--- Insertando elementos en el Arbol Binario ---
Arbol creado con exito.
Recorrido Inorder (debe estar ordenado de menor a mayor):
20 30 40 50 60 70 80
PS C:\Users\User\Desktop\important\reportes> █
```

Figure 12: Execution output showing insertion and inorder traversal of the Binary Search Tree.